

Nano Processor Design

Group 40

Team Details

Senanayaka H.M.S.	240605B
Senanayaka S.V.	240606E
Sharaf MFM	240619V
Sheereen Humaa M.R.	240621U

Introduction

The objective of this lab was to create a simple 4-bit Nano processor capable of executing basic arithmetic instructions and conditional jump operations. The processor was built using VHDL, simulated, and tested on the BASYS 3 FPGA board. Key components used to build this processor:

1. Add / Subtract unit

Performs addition and subtraction using 2's complement numbers.

2. Program Counter (PC)

A 3-bit counter that keeps track of which instructions to execute next. It can be reset, incremented, or jump.

3. Adder

Used to increment the Program Counter.

4. Multiplexers (MUXes)

Used to select between different data inputs and control signals.

5. Register Bank

Holds 8 registers (R0 to R7) that store 4-bit data values. Register R0 is always fixed at zero.

6. Buses

Shared data lines used to connect components and transfer information.

7. Program ROM

Stores the binary instructions that the processor executes.

8. Instruction Decoder

Reads each 12-bit binary instruction and activates the correct components to carry it out. These instructions are written in a simplified assembly language and then translated into binary format for execution.

Instruction Set

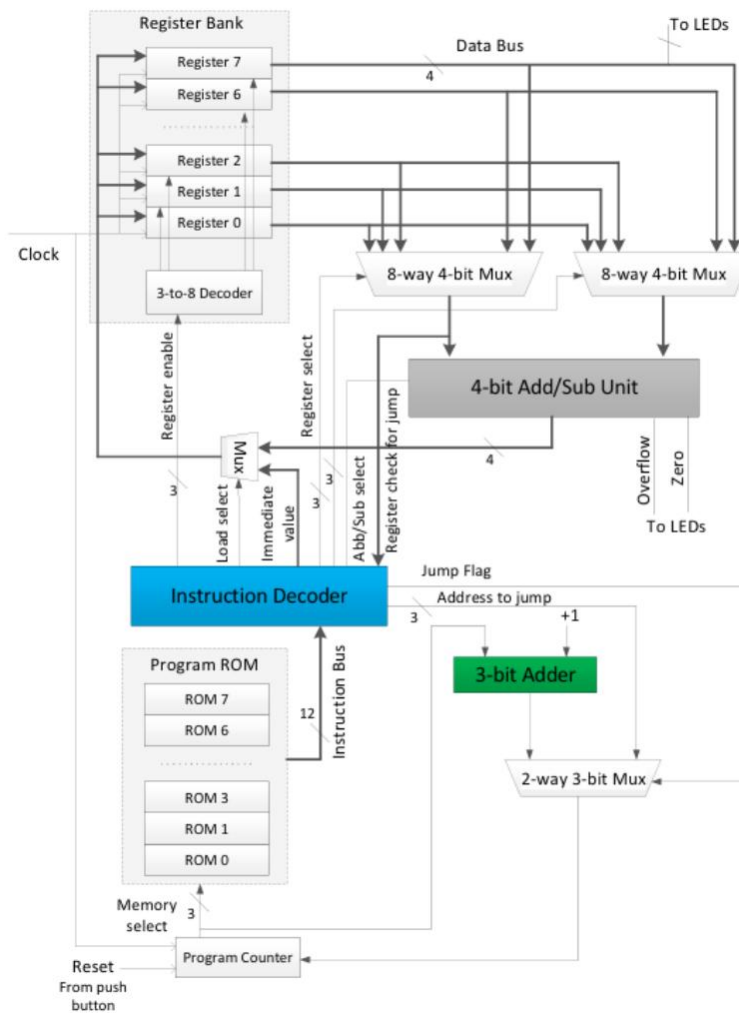
Instruction	Description	12-bit Format
MOVI R, d	Move immediate value d to register R ($R \leftarrow d$) $R \in [0, 7], d \in [0, 15]$	1 0 R R R 0 0 0 d d d d
ADD Ra, Rb	Add values in registers Ra and Rb and store the result in Ra ($Ra \leftarrow Ra + Rb$) $Ra, Rb \in [0, 7]$	0 0 Ra Ra Ra Rb Rb Rb 0 0 0 0
NEG R	2's complement of registers R ($R \leftarrow -R$) $R \in [0, 7]$	0 1 R R R 0 0 0 0 0 0 0
JZR R, d	Jump if value in register R is 0 If $R == 0$ $PC \leftarrow d$; Else $PC \leftarrow PC + 1$ $R \in [0, 7], d \in [0, 7]$	1 1 R R R 0 0 0 0 d d d

Instruction Encoding

Instruction Type	Op code (2 bits)
ALU Operations	00
NEG	01
MOVI	10
JZR	11

ALU Operation	Operation code	Sub-opcode
ADD	000	0000
SUB	001	0001
NEG	010	-
AND	011	0010
OR	100	0011
XOR	101	0100
MUL	110	0101
CMP	111	0111

High level architecture of the Nano processor



Core Components

Add / Subtract Unit

Design Source Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.cpu_types.all;

entity Add_Sub_4_bit is
    Port ( A_AS   : in  STD_LOGIC_VECTOR (3 DOWNT0 0);
          B_AS   : in  STD_LOGIC_VECTOR (3 DOWNT0 0);
          CTRL    : in  STD_LOGIC;
          S_AS    : out STD_LOGIC_VECTOR (3 DOWNT0 0);
          Zero    : out STD_LOGIC;
          OverFlow : out STD_LOGIC);
end Add_Sub_4_bit;

architecture Structural of Add_Sub_4_bit is
    signal B_inter : STD_LOGIC_VECTOR(3 DOWNT0 0);
    signal S_inter : STD_LOGIC_VECTOR(3 DOWNT0 0);
begin
    B_inter(0) <= B_AS(0) XOR CTRL;
    B_inter(1) <= B_AS(1) XOR CTRL;
    B_inter(2) <= B_AS(2) XOR CTRL;
    B_inter(3) <= B_AS(3) XOR CTRL;

    RCA_inst : entity work.RCA_4
        port map(A => A_AS, B => B_inter, C_in => CTRL, S => S_inter, C_out => OverFlow);

    S_AS <= S_inter;
    Zero <= '1' when S_inter = "0000" else '0';
end Structural;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.cpu_types.all;
```

entity AU is

```
Port( I1      : in data_bus;  
      I2      : in data_bus;  
      O       : out data_bus;  
      Overflow : out std_logic;  
      Zero     : out std_logic;  
      Operation : in Operation_Sel);
```

end AU;

architecture Behavioral of AU is

```
signal AS_CTRL : std_logic;
```

begin

```
process(Operation) begin
```

```
case Operation is
```

```
when AU_SUB_SIGNAL => AS_CTRL <= '1';
```

```
when others => AS_CTRL <= '0';
```

```
end case;
```

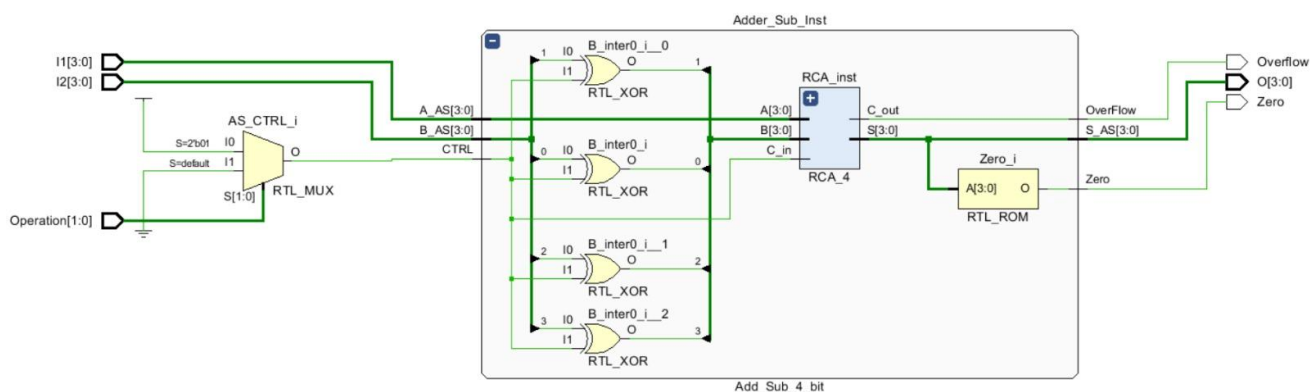
```
end process;
```

```
Adder_Sub_Inst : entity work.Add_Sub_4_bit
```

```
port map(A_AS => I1, B_AS => I2, CTRL => AS_CTRL, S_AS => O, Zero => Zero, OverFlow => Overflow);
```

```
end Behavioral;
```

Elaborated Design Schematic



Simulation Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.cpu_types.all;

entity TB_AU is
end TB_AU;

architecture Behavioral of TB_AU is
    signal I1, I2, O    : data_bus;
    signal Overflow, Zero : std_logic;
    signal Operation     : Operation_Sel;
begin
    UUT: entity work.AU
        port map(
            I1 => I1, I2 => I2, O => O, Overflow => Overflow,
            Zero => Zero, Operation => Operation
        );

    process
    begin
        Operation <= AU_ADD_SIGNAL;
        I1 <= "0011"; I2 <= "0010";
        wait for 200 ns;

        Operation <= AU_SUB_SIGNAL;
        I1 <= "0101"; I2 <= "0001";
        wait for 200 ns;

        Operation <= AU_SUB_SIGNAL;
        I1 <= "0010"; I2 <= "0010";
        wait for 200 ns;

        Operation <= AU_ADD_SIGNAL;
        I1 <= "1111"; I2 <= "0001";
        wait for 200 ns;
```

```

wait;
end process;
end Behavioral;

```

Timing Diagram



Program Counter

Design Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity PC is
    Port ( A : in STD_LOGIC_VECTOR(2 downto 0);
          Res : in STD_LOGIC;
          Clk : in STD_LOGIC;
          M : out std_logic_vector(2 downto 0));
end PC;

architecture Behavioral of PC is

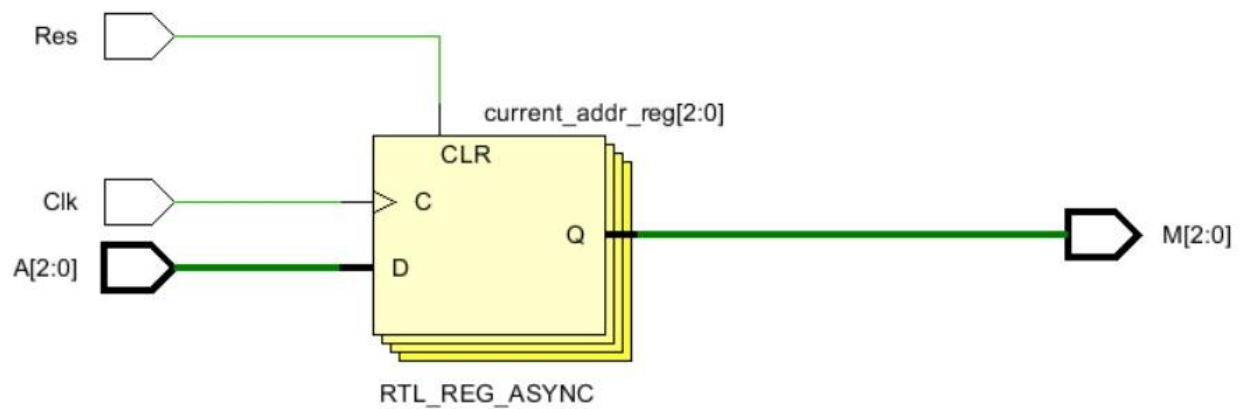
```

```

    signal current_addr : unsigned(2 downto 0) := "000";
begin
    process(Clk, Res) begin
        if Res = '1' then
            current_addr <= (others => '0');
        elsif rising_edge(Clk) then
            current_addr <= unsigned(A);
        end if;
    end process;
    M <= std_logic_vector(current_addr);
end Behavioral;

```

Elaborated Design Schematic



Simulation Source File

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity TB_PC is
end TB_PC;

```

```

architecture Behavioral of TB_PC is

```

```

    signal A : std_logic_vector(2 downto 0);

```



```

signal Res : std_logic;
signal Clk : std_logic := '0';
signal M : std_logic_vector(2 downto 0);
constant clk_period : time := 10 ns;

begin

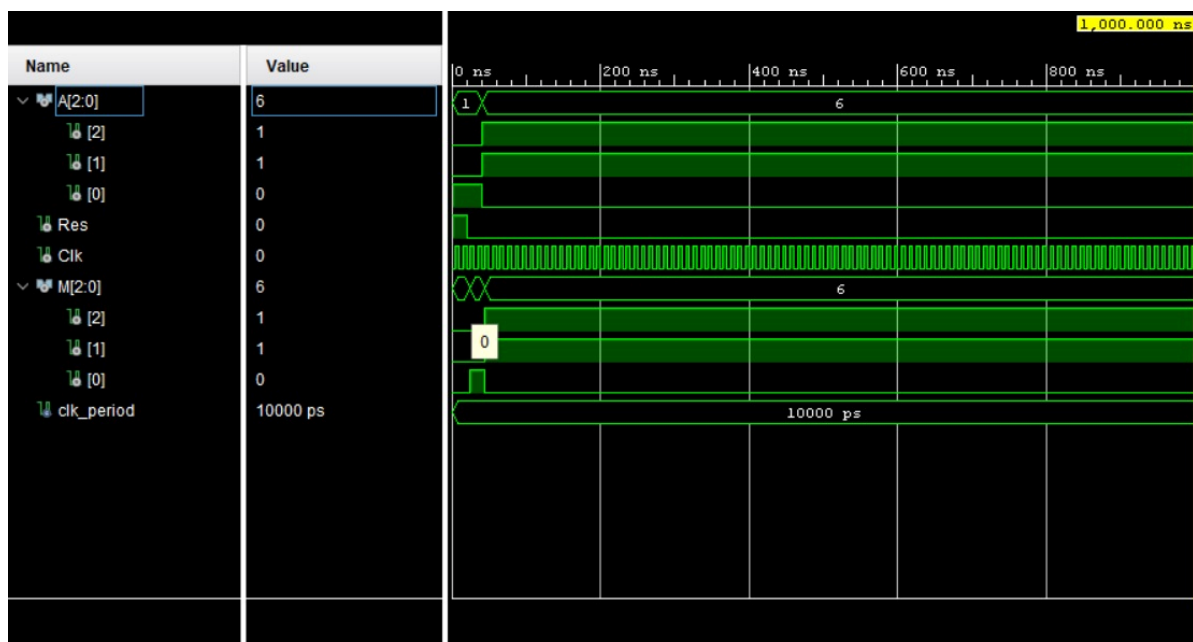
  UUT: entity work.PC
    port map (A => A, Res => Res, Clk => Clk, M => M);

  Clk_process : process
  begin
    Clk <= '0'; wait for clk_period/2;
    Clk <= '1'; wait for clk_period/2;
  end process;

  process
  begin
    Res <= '1'; A <= "001"; wait for 20 ns;
    Res <= '0'; wait for 20 ns;
    A <= "110"; wait for 20 ns;
    wait;
  end process;
end Behavioral;

```

Timing Diagram



Address Selector

Design Source File

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity Address_Selector is
```

```
    Port ( PC : in std_logic_vector(2 downto 0);
```

```
          JA : in std_logic_vector(2 downto 0);
```

```
          J  : in std_logic;
```

```
          A  : out std_logic_vector(2 downto 0));
```

```
end Address_Selector;
```

```
architecture Behavioral of Address_Selector is
```

```
begin
```

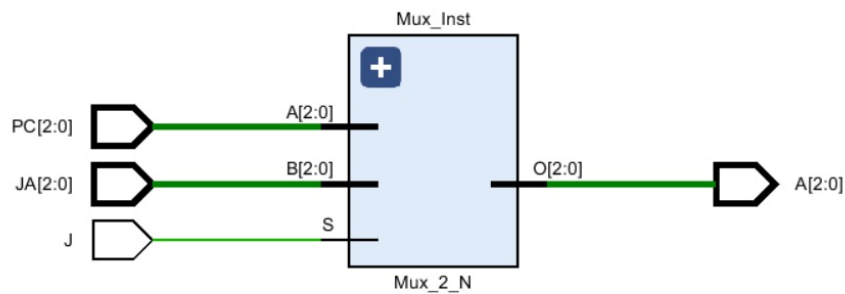
```
    Mux_Inst: entity work.Mux_2_N
```

```
        generic map (N => 3)
```

```
        port map (S => J, A => PC, B => JA, O => A);
```

```
end Behavioral;
```

Elaborated Design Schematic



Simulation Source File

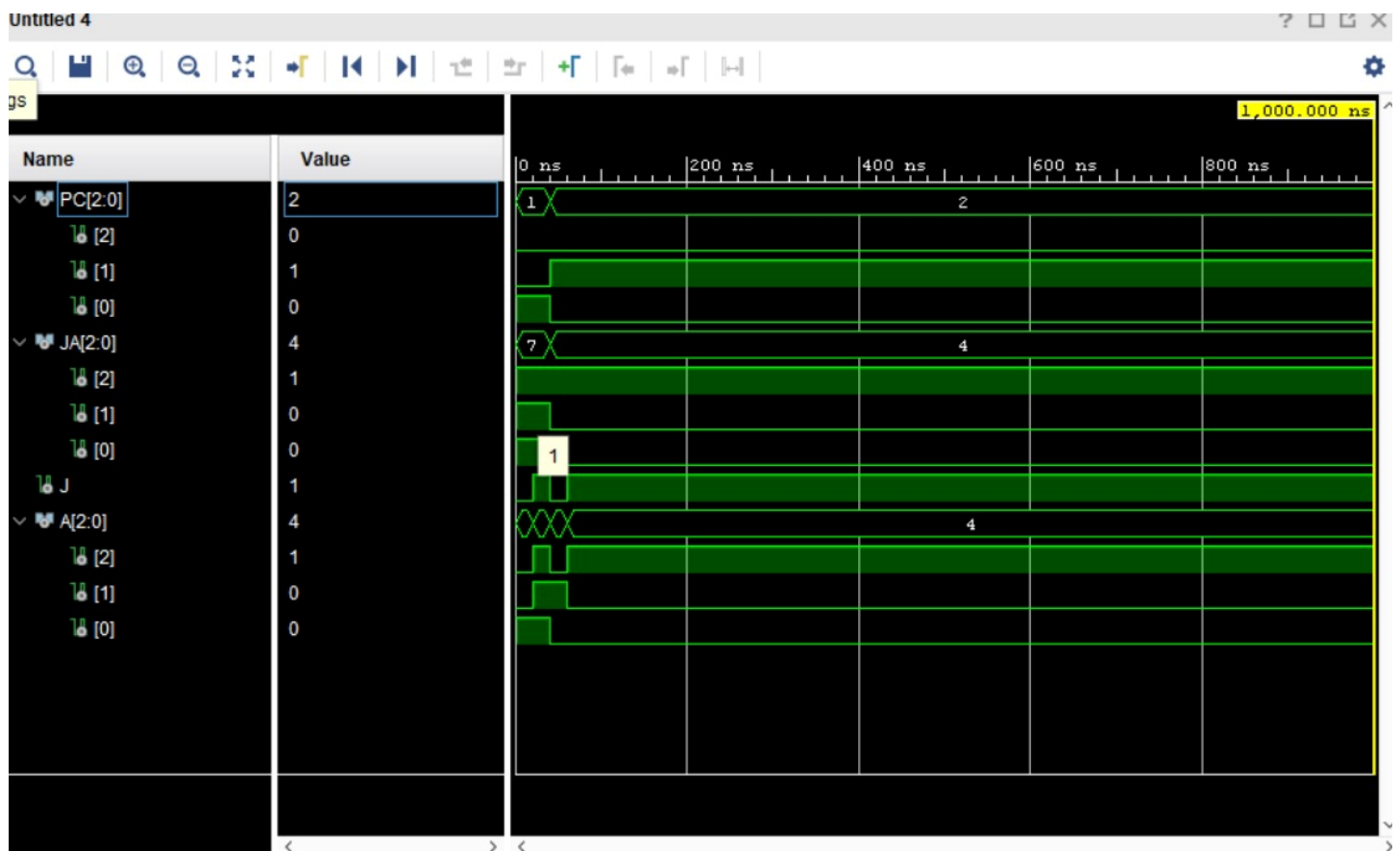
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity TB_Address_Selector is
end TB_Address_Selector;

architecture Behavioral of TB_Address_Selector is
    signal PC : std_logic_vector(2 downto 0);
    signal JA : std_logic_vector(2 downto 0);
    signal J : std_logic;
    signal A : std_logic_vector(2 downto 0);
begin
    UUT: entity work.Address_Selector
        port map (PC => PC, JA => JA, J => J, A => A);

    process
    begin
        PC <= "001"; JA <= "111"; J <= '0'; wait for 20 ns;
        J <= '1'; wait for 20 ns;
        PC <= "010"; JA <= "100"; J <= '0'; wait for 20 ns;
        J <= '1'; wait for 20 ns;
        wait;
    end process;
end Behavioral;
```

Timing Diagram



Load Selector

Design Source File

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

package constants is

 constant Immediate_Load : std_logic := '1';

end package constants;

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use work.cpu_types.all;

use work.constants.all;

entity Load_Selector is

 port(LS : in std_logic; IM : in data_bus; R : in data_bus; O : out data_bus);

end Load_Selector;

architecture Behavioral of Load_Selector is

```
signal A, B : data_bus;
```

```
begin
```

```
IM_LOAD_GEN : if Immediate_Load = '0' generate
```

```
    A <= IM; B <= R;
```

```
end generate IM_LOAD_GEN;
```

```
REG_LOAD_GEN : if Immediate_Load = '1' generate
```

```
    A <= R; B <= IM;
```

```
end generate REG_LOAD_GEN;
```

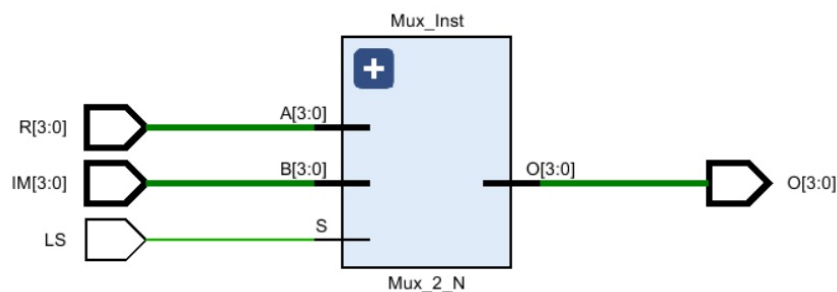
```
Mux_Inst: entity work.Mux_2_N
```

```
generic map(N => 4)
```

```
port map(S => LS, A => A, B => B, O => O);
```

```
end Behavioral;
```

Elaborated Design Schematic



Simulation Source File

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use work.cpu_types.all;
```

```
entity TB_Load_Selector is
```

```
end TB_Load_Selector;
```

```
architecture Behavioral of TB_Load_Selector is
```

```

signal LS : std_logic;
signal IM : data_bus;
signal R : data_bus;
signal O : data_bus;

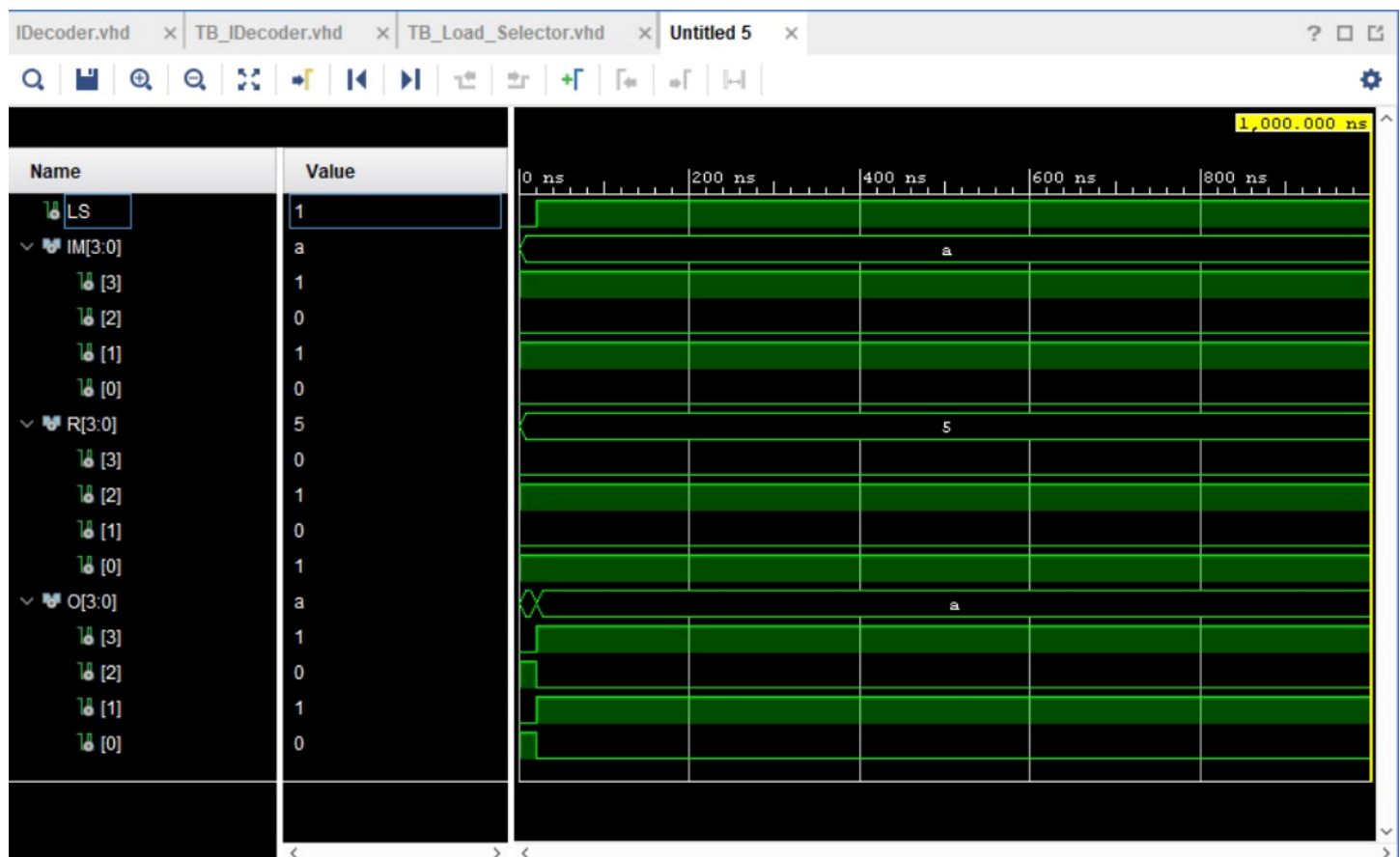
begin

  UUT: entity work.Load_Selector
    port map (LS => LS, IM => IM, R => R, O => O);

  process
  begin
    IM <= "1010"; R <= "0101";
    LS <= '0'; wait for 20 ns;
    LS <= '1'; wait for 20 ns;
    wait;
  end process;
end Behavioral;

```

Timing Diagram



PC Incrementer

Design Source File

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity PC_Inc is
```

```
    port( A_in : in  std_logic_vector(2 downto 0);
```

```
          A_out : out std_logic_vector(2 downto 0));
```

```
end PC_Inc;
```

```
architecture Behavioral of PC_Inc is
```

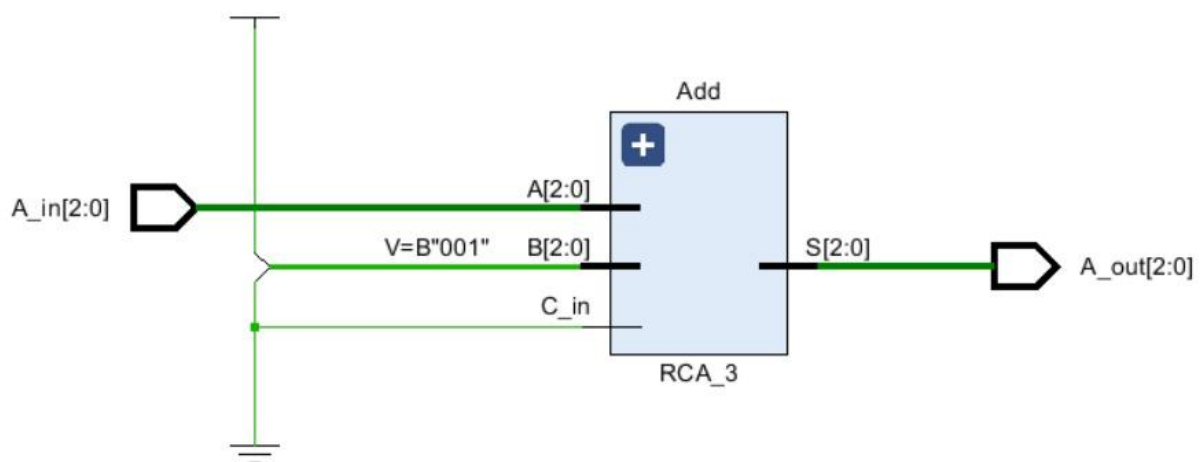
```
begin
```

```
    Add: entity work.RCA_3
```

```
        port map(A => A_in, B => "001", C_in => '0', S => A_out, C_out => open);
```

```
end Behavioral;
```

Elaborated Design Schematic



Simulation Source File

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;


entity TB_PC_Inc is

end TB_PC_Inc;


architecture Behavioral of TB_PC_Inc is

    signal A_in : std_logic_vector(2 downto 0);
    signal A_out : std_logic_vector(2 downto 0);
begin

    UUT: entity work.PC_Inc
        port map (A_in => A_in, A_out => A_out);


    process
    begin
        A_in <= "000"; wait for 20 ns;
        A_in <= "011"; wait for 20 ns;
        A_in <= "111"; wait for 20 ns;
        wait;
    end process;
end Behavioral;
```


Timing Diagram



Program ROM

Design Source Code

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
use work.cpu_types.all;
```

```
entity Program_ROM is
```

```
    Port ( ROM_address : in  instruction_address;
```

```
          I           : out instruction_bus);
```

```
end Program_ROM;
```

```
architecture Behavioral of Program_ROM is
```

```
    type rom_type is array (0 to 7) of instruction_bus;
```

```
    constant PROGRAM : rom_type := (
```

```
        0 => "101110000001", -- MOVI R7, 1 (Output becomes 1)
```

```
        1 => "100010000010", -- MOVI R1, 2
```

```
        2 => "001110010000", -- ADD R7, R1 (Output becomes 1+2 = 3)
```

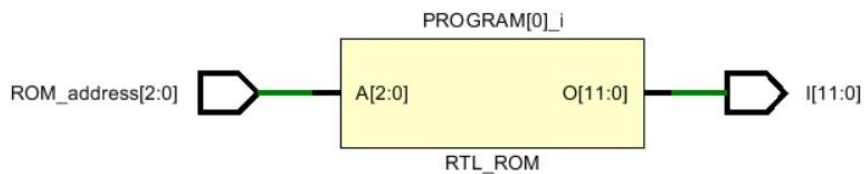
```
        3 => "100100000011", -- MOVI R2, 3
```

```

4 => "001110100000", -- ADD R7, R2 (Output becomes 3+3 = 6) <-- FIXED HERE
5 => "110000000101", -- JZR R0, 5 (Infinite loop to hold the result)
6 => "000000000000",
7 => "000000000000"
);
begin
  process(ROM_address) begin
    I <= PROGRAM(to_integer(unsigned(ROM_address)));
  end process;
end Behavioral;

```

Elaborated Design Schematic



Simulation Source Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.cpu_types.all;

```

```

entity TB_Program_ROM is
end TB_Program_ROM;

```

```

architecture Behavioral of TB_Program_ROM is
  signal ROM_address : instruction_address;
  signal I           : instruction_bus;
begin
  UUT: entity work.Program_ROM

```

```
port map (ROM_address => ROM_address, I => I);
```

```
process
```

```
begin
```

```
    ROM_address <= "000"; wait for 200 ns;
```

```
    ROM_address <= "001"; wait for 200 ns;
```

```
    ROM_address <= "010"; wait for 200 ns;
```

```
    ROM_address <= "101"; wait for 200 ns;
```

```
    ROM_address <= "111"; wait for 200 ns;
```

```
    wait;
```

```
end process;
```

```
end Behavioral;
```

Timing Diagram



Reg Design

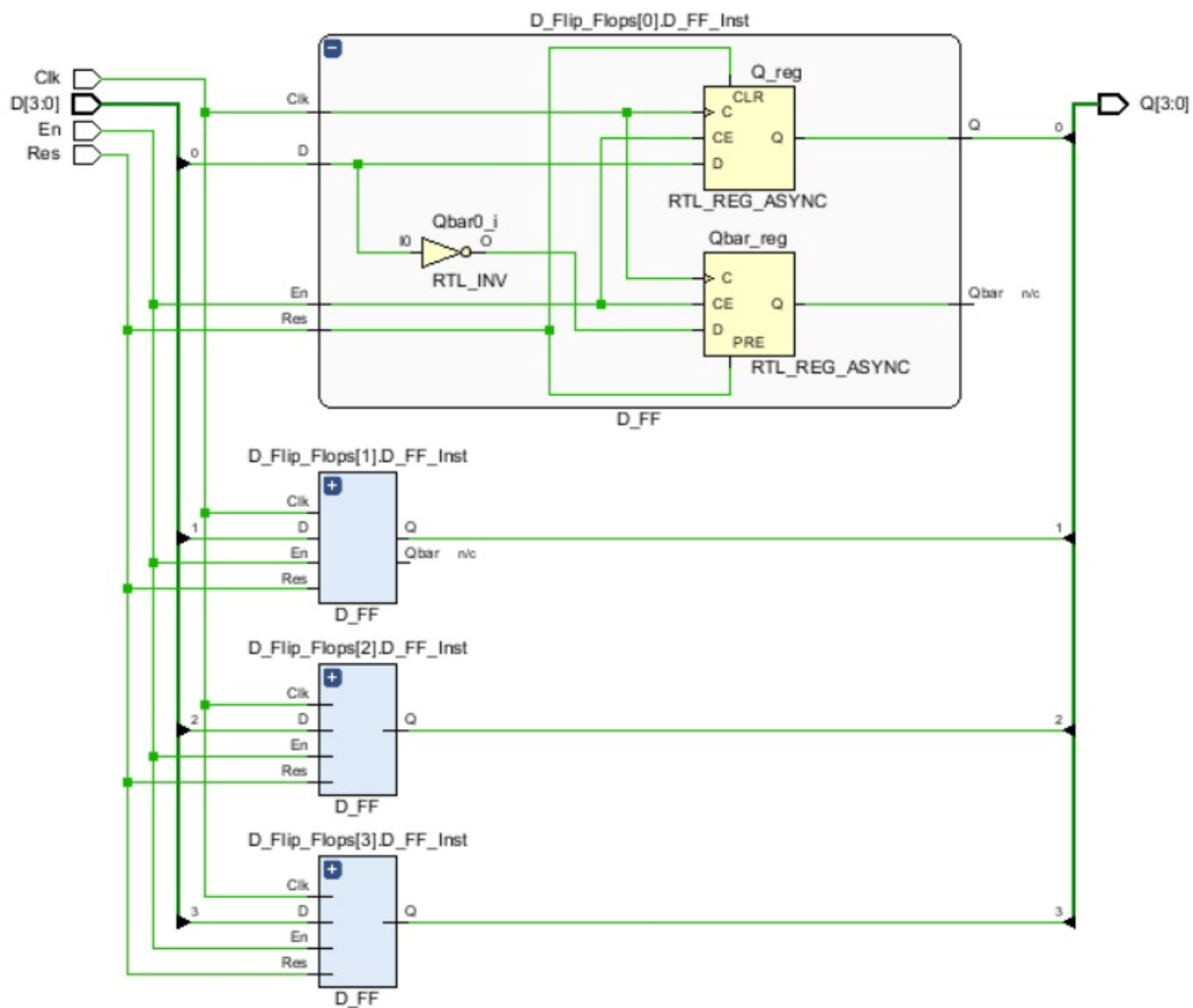
Design Source Code

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Reg is
    generic( N : integer := 4 );
    Port ( D : in std_logic_vector(N-1 downto 0);
          Res : in STD_LOGIC;
          En : in STD_LOGIC;
          Clk : in STD_LOGIC;
          Q : out std_logic_vector(N-1 downto 0));
end Reg;

architecture Behavioral of Reg is
begin
    D_Flip_Flops: for i in 0 to N-1 generate
        D_FF_Inst: entity work.D_FF port map(
            D => D(i),
            Res => Res,
            En => En,
            Clk => Clk,
            Q => Q(i),
            Qbar => open
        );
    end generate D_Flip_Flops;
end Behavioral;
```

Elaborated Design Schematic



Simulation Source File

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity TB_Reg is

end TB_Reg;

architecture Behavioral of TB_Reg is

signal D : std_logic_vector(3 downto 0);

signal Res : std_logic := '0';

signal En : std_logic := '0';

signal Clk : std_logic := '0';

signal Q : std_logic_vector(3 downto 0);

constant clk_period : time := 100 ns;

begin

UUT: entity work.Reg

generic map (N => 4)

port map (D => D, Res => Res, En => En, Clk => Clk, Q => Q);

Clk_process : process

begin

Clk <= '0'; wait for clk_period/2;

Clk <= '1'; wait for clk_period/2;

end process;

process

begin

Res <= '1'; wait for 200 ns;

Res <= '0';

D <= "1010"; En <= '0'; wait for 200 ns;

En <= '1'; wait for 200 ns;

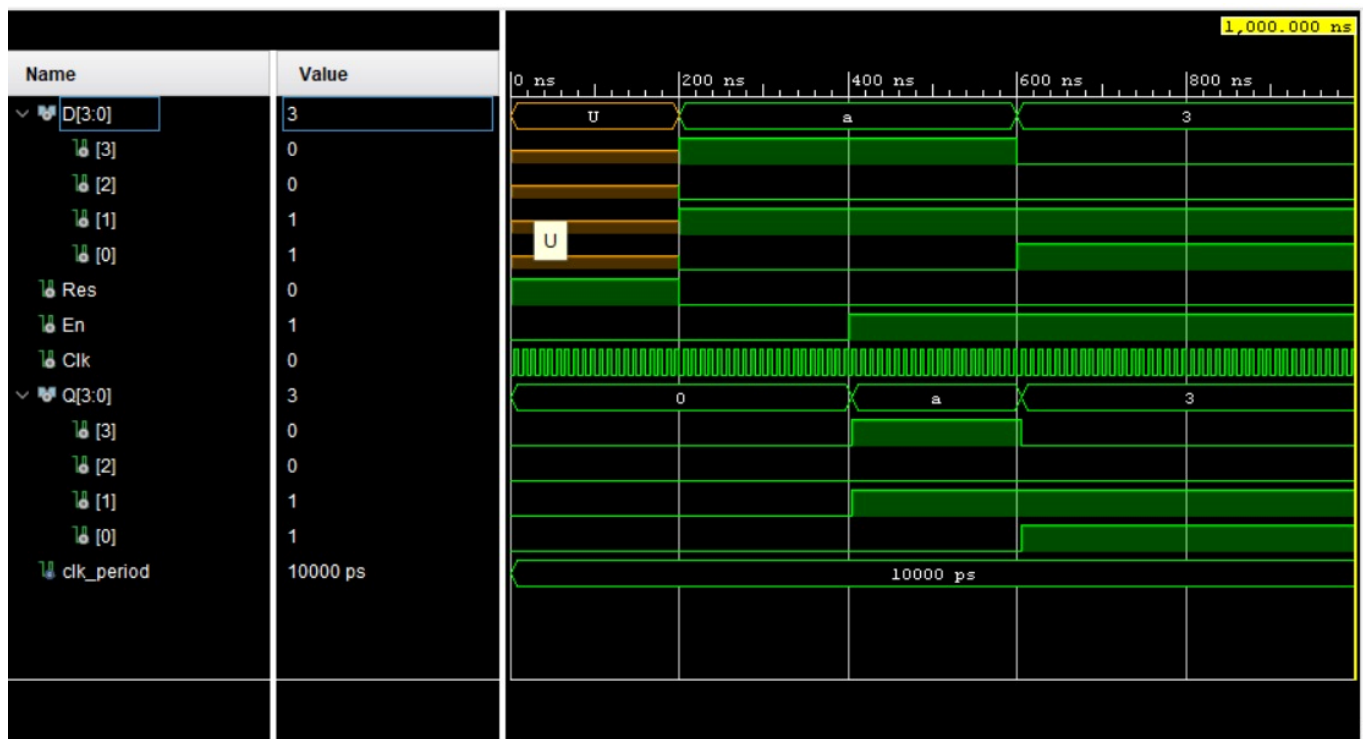
D <= "0011"; wait for 200 ns;

wait;

end process;

end Behavioral;

Timing Diagram



Register Bank

Design Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.cpu_types.all;

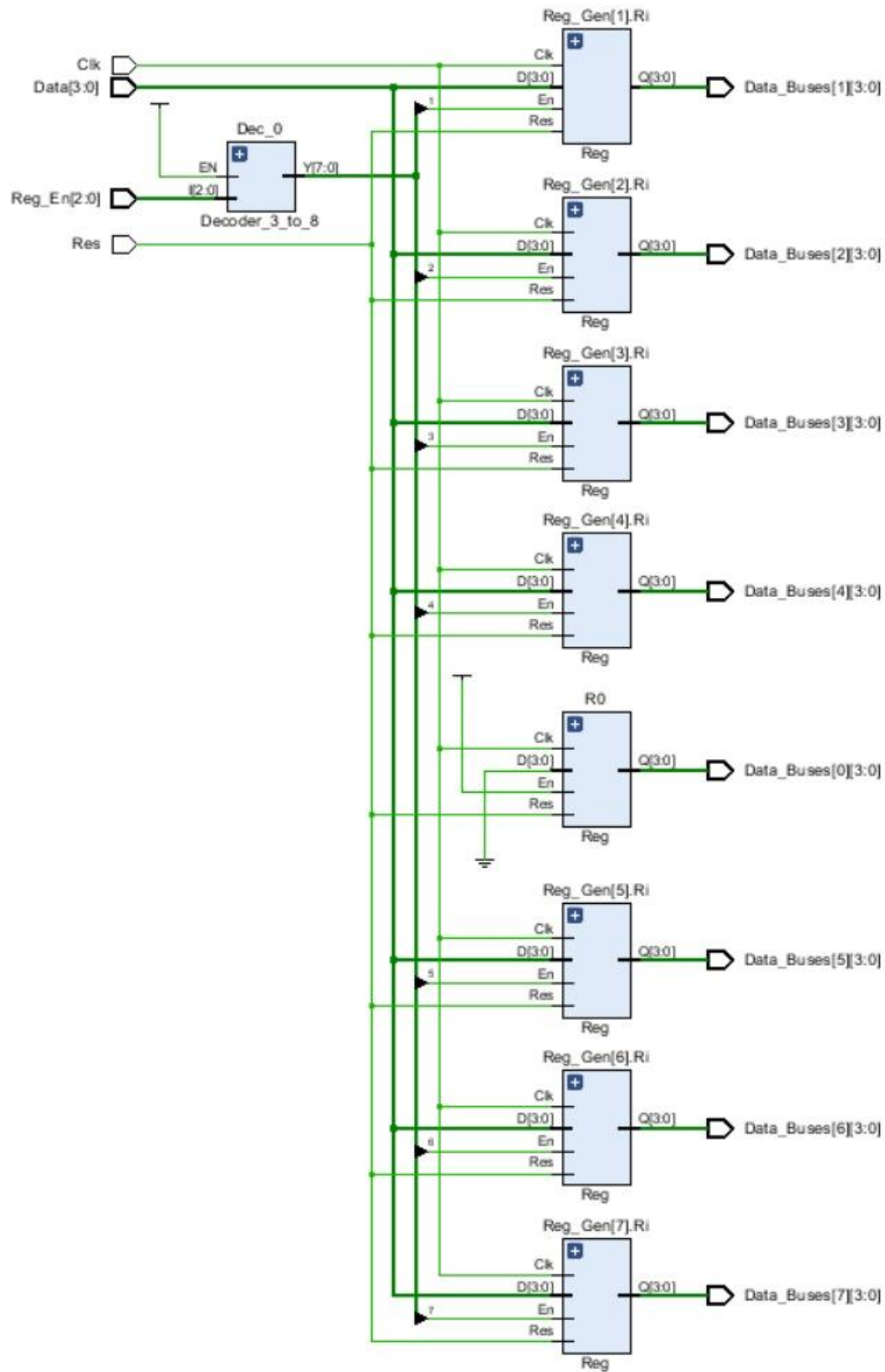
entity Register_Bank is
  Port ( Reg_En   : in  register_address;
        Res      : in  STD_LOGIC;
        Clk      : in  STD_LOGIC;
        Data     : in  data_bus;
        Data_Buses : out data_buses );
end Register_Bank;

architecture Behavioral of Register_Bank is
  signal Reg_sel : std_logic_vector(7 downto 0);
begin
  Dec_0 : entity work.Decoder_3_to_8
    port map(I => Reg_En, EN => '1', Y => Reg_sel);

  R0: entity work.reg generic map(N => 4)
    port map(D => "0000", Res => Res, En => '1', Clk => Clk, Q => Data_Buses(0));

  Reg_Gen: for i in 1 to 7 generate
    Ri: entity work.reg generic map(N => 4)
      port map(D => Data, Res => Res, En => Reg_sel(i), Clk => Clk, Q => Data_Buses(i));
  end generate;
end Behavioral;
```

Elaborated Design Schematic



Simulation Source File

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.cpu_types.all;

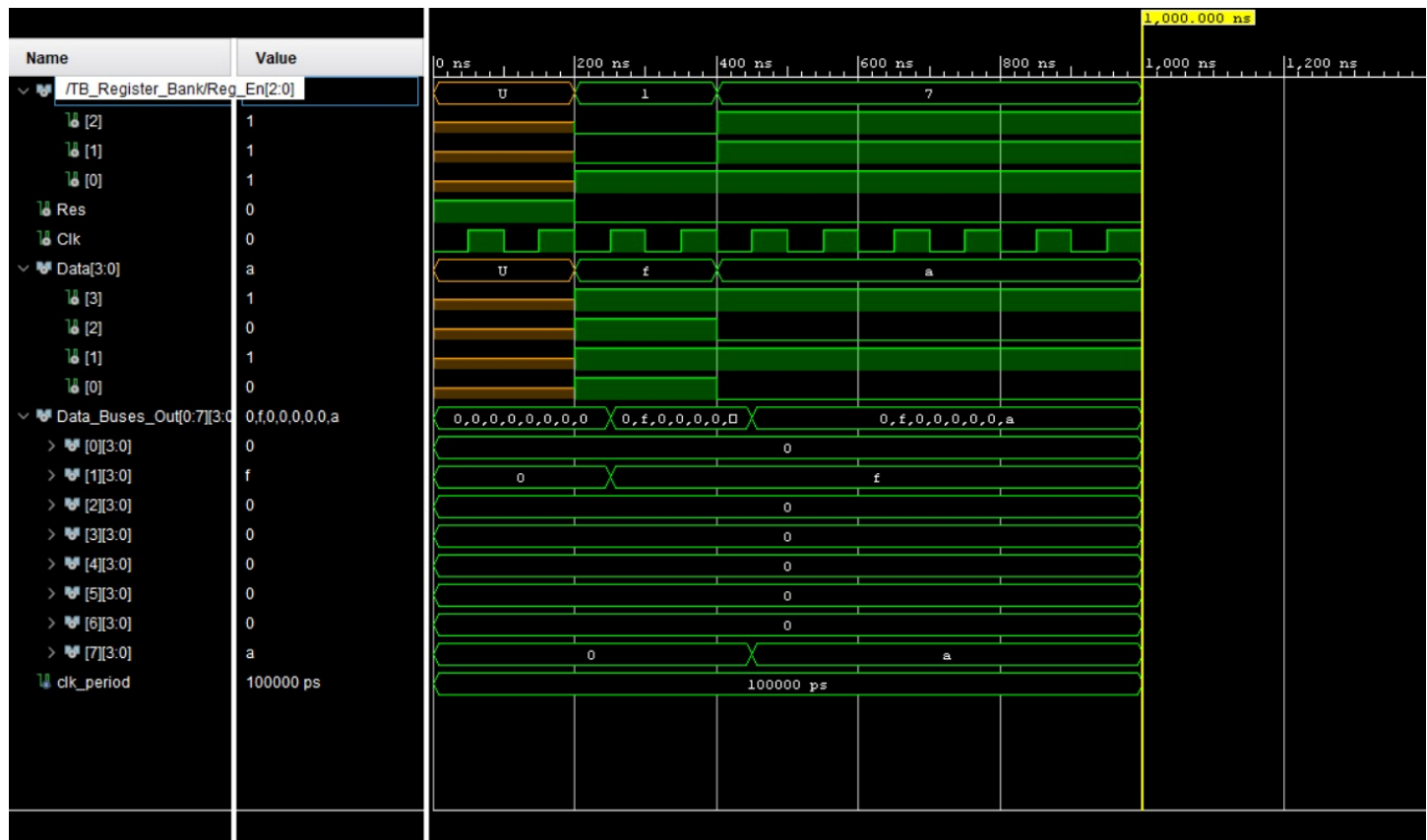
entity TB_Register_Bank is
end TB_Register_Bank;

architecture Behavioral of TB_Register_Bank is
    signal Reg_En : register_address;
    signal Res : std_logic;
    signal Clk : std_logic := '0';
    signal Data : data_bus;
    signal Data_Buses_Out : data_buses;
    constant clk_period : time := 100 ns;
begin
    UUT: entity work.Register_Bank
        port map(
            Reg_En => Reg_En, Res => Res, Clk => Clk,
            Data => Data, Data_Buses => Data_Buses_Out
        );

    Clk_process : process
    begin
        Clk <= '0'; wait for clk_period/2;
        Clk <= '1'; wait for clk_period/2;
    end process;

    process
    begin
        Res <= '1'; wait for 200 ns;
        Res <= '0';
        Data <= "1111"; Reg_En <= "001"; wait for 200 ns;
        Data <= "1010"; Reg_En <= "111"; wait for 200 ns;
        wait;
    end process;
end Behavioral;
```

Timing Diagram



Seven Segment LUT

Design Source Code

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
entity SevenSegmentLUT is
```

```
  Port ( address : in STD_LOGIC_VECTOR (3 downto 0);
```

```
        data : out STD_LOGIC_VECTOR (6 downto 0));
```

```
end SevenSegmentLUT;
```

```
architecture Behavioral of SevenSegmentLUT is
```

```
  type rom_type is array (0 to 15) of std_logic_vector(6 downto 0);
```

```
  signal sevenSegment_ROM : rom_type := (
```

```
    "1000000", "1111001", "0100100", "0110000", "0011001", "0010010", "0000010", "1111000",
```

```
    "0000000", "0010000", "0001000", "0000011", "1000110", "0100001", "0000110", "0001110"
```

```
  );
```

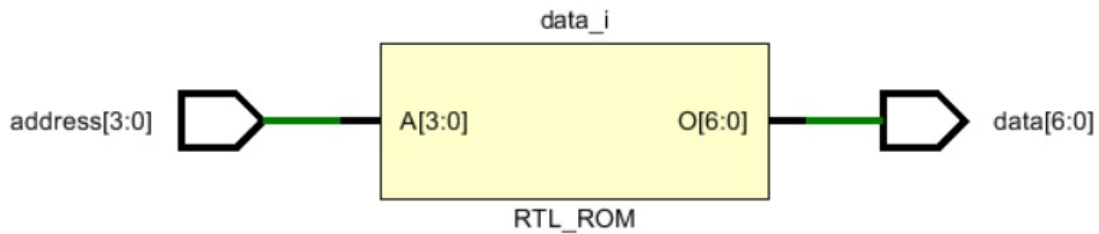
```
  begin
```

```

data <= sevenSegment_ROM(to_integer(unsigned(address)));
end Behavioral;

```

Elaborated Design Schematic



Simulation Source Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity TB_SevenSegmentLUT is
end TB_SevenSegmentLUT;

```

```

architecture Behavioral of TB_SevenSegmentLUT is

```

```

    signal address : std_logic_vector(3 downto 0);
    signal data    : std_logic_vector(6 downto 0);

```

```

begin

```

```

    UUT: entity work.SevenSegmentLUT
        port map (address => address, data => data);

```

```

process

```

```

begin

```

```

    address <= "0000"; wait for 200 ns;
    address <= "0011"; wait for 200 ns;
    address <= "0110"; wait for 200 ns;
    address <= "1010"; wait for 200 ns;
    address <= "1111"; wait for 200 ns;

```

```

wait;
end process;
end Behavioral;

```

Timing Diagram



Nano Processor

Design Source Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.cpu_types.all;

entity Nanoprocessor is
port(
    Clk    : in std_logic;
    Res    : in std_logic;
    Overflow : out std_logic;
    Zero    : out std_logic;
    Data    : out std_logic_vector(3 downto 0)
);

```

end Nanoprocessor;

architecture Behavioral of Nanoprocessor is

```
signal Next_Address    : std_logic_vector(2 downto 0);
signal Current_Address : std_logic_vector(2 downto 0);
signal Selected_Address : std_logic_vector(2 downto 0);
signal Jump_Address    : std_logic_vector(2 downto 0);
signal Jump_Flag       : std_logic;
signal Instruction      : std_logic_vector(11 downto 0);
```

```
signal Load_Selection  : std_logic;
signal Immediate_Value : std_logic_vector(3 downto 0);
signal OprASelect      : std_logic_vector(2 downto 0);
signal OprBSelect      : std_logic_vector(2 downto 0);
signal OprAData        : std_logic_vector(3 downto 0);
signal OprBData        : std_logic_vector(3 downto 0);
signal Operation_Res   : std_logic_vector(3 downto 0);
```

```
signal Register_Data   : data_buses;
signal AddSubSelect    : Operation_Sel;
signal Register_Enable : std_logic_vector(2 downto 0);
signal Selected_Load   : std_logic_vector(3 downto 0);
```

begin

```
Program_Counter : entity work.PC
  port map(A => Selected_Address, Clk => Clk, Res => Res, M => Current_Address);
```

```
PC_Incrementer : entity work.PC_Inc
  port map(A_in => Current_Address, A_out => Next_Address);
```

```
Address_Selector_0 : entity work.Address_Selector
  port map(PC => Next_Address, JA => Jump_Address, J => Jump_Flag, A => Selected_Address);
```

```
Program_ROM_0 : entity work.Program_ROM
  port map(ROM_address => Current_Address, I => Instruction);
```

Instruction_Decoder : entity work.IDecoder

```
port map(  
    I    => Instruction,  
    RCJump => OprAData, -- Fixed Datapath Bug!  
    REN   => Register_Enable,  
    RSA   => OprASelect,  
    RSB   => OprBSelect,  
    OpS   => AddSubSelect,  
    IM    => Immediate_Value,  
    J     => Jump_Flag,  
    JA    => Jump_Address,  
    L     => Load_Selection  
);
```

Load_Selector_0 : entity work.Load_Selector

```
port map(LS => Load_Selection, IM => Immediate_Value, R => Operation_Res, O => Selected_Load);
```

Opr_Selector_A : entity work.Mux_8_4

```
port map(S => OprASelect, D => Register_Data, EN => '1', Y => OprAData);
```

Opr_Selector_B : entity work.Mux_8_4

```
port map(S => OprBSelect, D => Register_Data, EN => '1', Y => OprBData);
```

Arithmetic_Unit_0 : entity work.AU

```
port map(I1 => OprAData, I2 => OprBData, O => Operation_Res, Overflow => Overflow,  
    Zero => Zero, Operation => AddSubSelect);
```

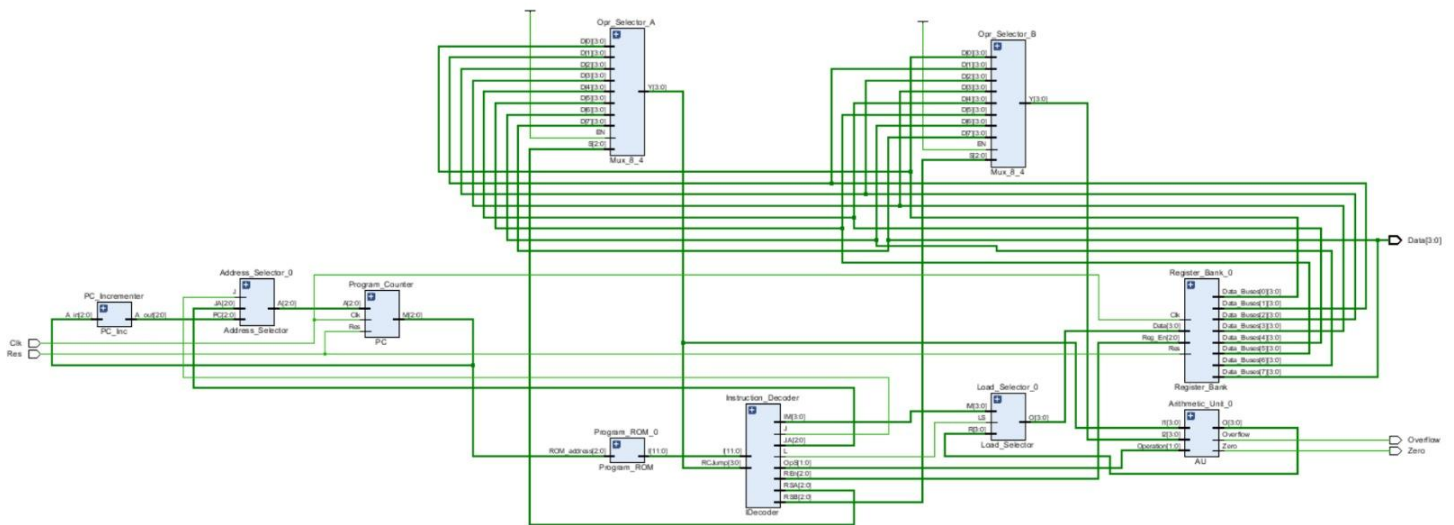
Register_Bank_0 : entity work.Register_Bank

```
port map(Reg_EN => Register_Enable, Res => Res, Clk => Clk, Data => Selected_Load,  
    Data_Buses => Register_Data);
```

```
Data <= Register_Data(7);
```

end Behavioral;

Elaborated Design Schematic



Simulation Source Code

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity Nanoprocessor_TB is

end Nanoprocessor_TB;

architecture Behavioral of Nanoprocessor_TB is

component Nanoprocessor

port(

Clk : in std_logic;

Res : in std_logic;

Overflow : out std_logic;

Zero : out std_logic;

Data : out std_logic_vector(31 downto 0)

```

    );

end component;


signal Clk    : std_logic := '0';
signal Res    : std_logic := '0';
signal Overflow : std_logic;
signal Zero   : std_logic;
signal Data   : std_logic_vector(3 downto 0);


constant clk_period : time := 100 ns;


begin


    UUT: Nanoprocessor
        port map (
            Clk => Clk,
            Res => Res,
            Overflow => Overflow,
            Zero => Zero,
            Data => Data
        );


    clk_process : process
    begin
        Clk <= '0';
        wait for clk_period/2;
        Clk <= '1';
        wait for clk_period/2;
    end process;
end;

```



```
end process;
```

```
stim_proc: process
```

```
begin
```

```
    Res <= '1';
```

```
    wait for 100 ns;
```

```
    Res <= '0';
```

```
    wait;
```

```
end process;
```

```
end Behavioral;
```

Timing Diagram



Optimized Codes

Updated AU

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use work.cpu_types.all;
```

```
entity Add_Sub_4_bit is
```

```
    Port ( A_AS   : in  STD_LOGIC_VECTOR (3 DOWNT0 0);
```

```
          B_AS   : in  STD_LOGIC_VECTOR (3 DOWNT0 0);
```

```
          CTRL   : in  STD_LOGIC; -- 0 for Add, 1 for Sub
```

```
          S_AS   : out STD_LOGIC_VECTOR (3 DOWNT0 0);
```

```
          Zero   : out STD_LOGIC;
```

```
          OverFlow : out STD_LOGIC);
```

```
end Add_Sub_4_bit;
```

```
architecture Structural of Add_Sub_4_bit is
```

```
    signal B_inter : STD_LOGIC_VECTOR(3 DOWNT0 0);
```

```
    signal S_inter : STD_LOGIC_VECTOR(3 DOWNT0 0);
```

```
begin
```

```
    B_inter(0) <= B_AS(0) XOR CTRL;
```

```
    B_inter(1) <= B_AS(1) XOR CTRL;
```

```
    B_inter(2) <= B_AS(2) XOR CTRL;
```

```
    B_inter(3) <= B_AS(3) XOR CTRL;
```

```
    CLA_inst : entity work.CLA_4
```

```
        port map(
```

```
            A    => A_AS,
```

```

    B    => B_inter,
    C_in => CTRL,
    S    => S_inter,
    C_out => OverFlow
);

```

```

S_AS <= S_inter;

Zero <= '1' when S_inter = "0000" else '0';

end Structural;

```

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use work.cpu_types.all;

```

```

entity AU is
    Port( I1    : in data_bus;
          I2    : in data_bus;
          O     : out data_bus;
          Overflow : out std_logic;
          Zero   : out std_logic;
          Operation : in Operation_Sel);
end AU;

```

```

architecture Behavioral of AU is
    signal AS_CTRL : std_logic;

begin
    process(Operation) begin
        case Operation is

```

```

    when AU_SUB_SIGNAL => AS_CTRL <= '1';
    when others => AS_CTRL <= '0';
end case;
end process;

```

Adder_Sub_Inst : entity work.Add_Sub_4_bit

```

port map(
    A_AS  => I1,
    B_AS  => I2,
    CTRL  => AS_CTRL,
    S_AS  => O,
    Zero  => Zero,
    OverFlow => Overflow
);

```

end Behavioral;

Updated PC Incrementer

```

library IEEE;

```

```

use IEEE.STD_LOGIC_1164.ALL;

```

entity PC_Inc is

```

port( A_in : in std_logic_vector(2 downto 0);
      A_out : out std_logic_vector(2 downto 0));

```

end PC_Inc;

architecture Behavioral of PC_Inc is

begin

```

    Add: entity work.CLA_3

```

```
port map(  
    A    => A_in,  
    B    => "001", -- Increment by 1  
    C_in => '0',  
    S    => A_out,  
    C_out => open  -- Carry out is not needed for a 3-bit PC  
);  
end Behavioral;
```

Constraint

Clock Signal

```
set_property PACKAGE_PIN W5 [get_ports {Clk}]
set_property IOSTANDARD LVCMOS33 [get_ports {Clk}]
create_clock -period 10.000 -name sys_clk -waveform {0 5} [get_ports {Clk}]
```

Reset Button

```
set_property PACKAGE_PIN T18 [get_ports {Res}]
set_property IOSTANDARD LVCMOS33 [get_ports {Res}]
```

Output LEDs (Data_tb[3:0])

```
set_property PACKAGE_PIN U16 [get_ports {Data[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[0]}]
```

```
set_property PACKAGE_PIN E19 [get_ports {Data[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[1]}]
```

```
set_property PACKAGE_PIN U19 [get_ports {Data[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[2]}]
```

```
set_property PACKAGE_PIN V19 [get_ports {Data[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {Data[3]}]
```

Overflow and Zero flags to LEDs

```
set_property PACKAGE_PIN W19 [get_ports {Overflow}]
set_property IOSTANDARD LVCMOS33 [get_ports {Overflow}]
```

```
set_property PACKAGE_PIN U14 [get_ports {Zero}]
set_property IOSTANDARD LVCMOS33 [get_ports {Zero}]
```

Optimization

In this project, a Carry Lookahead Adder (CLA) was implemented to perform the increment operation of the Program Counter (PC) by one, replacing the conventional Ripple Carry Adder (RCA). The CLA was selected due to its ability to generate carry signals more efficiently, thereby minimizing propagation delay and improving the overall speed of the Arithmetic Logic Unit (ALU).

Conclusion

In this laboratory exercise, a 4-bit Nano processor was designed and implemented to execute four basic instructions: MOVI, ADD, NEG, and JZR. The processor architecture consisted of multiple interconnected modules such as the Add/Subtract Unit, Program Counter, Register Bank, Multiplexers, Instruction Decoder, and Program ROM, each contributing to the overall operation of the system.

Key Achievements

- The Nano processor was able to execute assembly level programs, including arithmetic operations, data transfers and conditional jumps.
- The implementation of the Carry Lookahead Adder (CLA) contributed to reducing the critical path delay, thereby improving the overall processing speed and performance of the system.

Team Contribution

Name	Contribution
Senanayaka H.M.S 240605B	HA, FA, RCA, AU – 2.5 hours Simulations for above – 30 minutes Final Checkup on board – 15 minutes Constraint file – 30 minutes Debugging – 30 minutes
Senanayaka S.V. 240606E	MUX, Load selector – 1.5 hours Slow clock, Seven Segment LUT – 1.5 hours Program Rom – 1 hour Simulations for above – 30 minutes Interconnecting components and simulation – 3 hours
Sharaf M.F.M. 240619V	Program counter + incrementer – 2 hours Address Selector, Instruction Decoder – 2 hours Simulations for above – 30 minutes Debugging – 30 minutes Capturing Timing Diagram and Screenshots – 1 hour
Sheereen Humaa M.R. 240621U	D flip-flop, register – 1.5 hours Register Bank, Decoders – 1.5 hours Simulations for above – 30 minutes Final Lab Report – 3 hours Presentation Slides – 1.5 hours